

Please incorporate the following tables and columns. Determine the suitable data types for any new columns along with foreign key. Ensure that any necessary data is added to achieve the desired output.

Customers:

Customer_id

First_name

Last_name

Email

Phone

Address

City

State_province

Country

Postal_code

Date_of_birth

Gender

Employees:

Employee_id

First_name

Last_name

Department_id

Salary

Departments:

Department_id

Department_name

Salary_History:

Sync_History_id

Employee_id

Salary

Effective_date

Products:

Product_id

Product_name
Category_name
Unit_price - 10
Featured

Promotions:
Promotion_id
Product_id
Promotion_name
Start_date
End_date
DiscountAmount - 2
Active

Orders:
Order_id
Promotion_id
Product_id
Quantity 2
Customer_id
Order_date
Price - 8

Category_Summary:
Category_summary_id
Category_name
Revenue

1. RANK(): Rank employees within each department based on their Salary.
2. DENSERANK(): Determine the top three performing customers based on orders placed in current month and assign a dense rank to them.
3. ROWNUMBER(): Generate a query that includes the total number of orders for each customer, assigning a row number to each order within the customer's record.

4. Subquery with Insert, Edit, Delete, Update:
 - a. Insert new records into a 'Promotions' table for any 5 products.
 - b. Trigger: Update the 'Products' table where the product's order is placed more than thrice in the current month to mark these product as 'Featured'.
 - c. Ensure to delete any promotions created (Start_date) before last 6 months.
5. Create a view that joins the 'Orders' and 'Customers' tables to display order details along with customer information.
6. Build a complex view that joins multiple tables such as 'Orders', 'Products' to display detailed order data along with product information for last 3 months.
7. View with WITH CHECK OPTION: Implement a view that shows employee salaries and allows for salary updates. Utilize the 'WITH CHECK OPTION' to ensure that salary must be > 10000.
8. User-defined Functions: Develop a function that calculates the total cost of an order based on the quantity and unit price of each product.
9. WHILE Loop with Continue and Break:
 - a. Create a temp table named "#Numbers" with a single column named "Value" to store integer values.
 - b. Populate the "#Numbers" table with integers from 1 to 10.
 - c. Implement a WHILE loop to iterate through the "Numbers" table.
 - d. Within the loop, print each integer value.
 - e. If the integer value is even, skip to the next iteration using CONTINUE.
 - f. If the integer value is 5, break out of the loop using BREAK.
10. Cursor: Write a cursor to iterate through a 'Order' table and calculate the total revenue generated by each product category. Display the results and insert them in a separate summary table.
11. Normalize Unstructured Data into a Database Schema

Unstructured Data:

The unstructured data consists of the following fields in a spreadsheet:

Student ID
Name
Age
Gender
Address
Email
Phone Number
Course ID
Course Name
Instructor
Credit Hours
Grade

Task Instructions:

Analyze the unstructured data and identify entities, attributes, and relationships.

Apply normalization techniques (First Normal Form, Second Normal Form, Third Normal Form) to organize the data into a structured database schema.

Write the appropriate tables with primary keys, foreign keys, and relationships to represent the normalized schema.

Document the normalization process, including the steps taken and any assumptions made.

Present the normalized schema and the documentation explaining the normalization process.

Additional Considerations:

Ensure that foreign key relationships are properly defined between related tables.

Pay attention to data integrity constraints such as primary keys, unique constraints, and foreign key constraints.

Optimize the schema for efficiency and ease of use.

Consider any additional features or requirements that may arise during the normalization process and make appropriate adjustments to the schema.